

Microprocesadores – Actividad V

Transmisión de Datos vía Bluetooth BLE

Alumno: Tapia Ortega Citlalli Yael

Profesor: José Luis Barbosa Pacheco

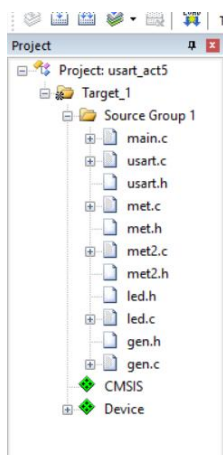
Fecha: 17/11/2025

Resumen

En esta actividad se desarrolló un sistema modular basado en el microcontrolador STM32F103C8, en el cual se generaron 200 números pseudoaleatorios almacenados en la memoria SRAM y posteriormente transmitidos mediante el periférico USART1 hacia un módulo Bluetooth HM-10 compatible con BLE. Se diseñaron dos modos de operación que permiten distintas formas de transmisión y se verificó el funcionamiento mediante un dispositivo móvil utilizando la aplicación Tiny BLE Serial.

I. Introducción

El propósito de esta actividad es realizar un sistema capaz de generar datos en la memoria SRAM, procesarlos y transmitirlos



mediante comunicación serial hacia un módulo Bluetooth BLE HM-10. Para lograrlo se empleó un código base dado por el profesor modificando la arquitectura al dividir el programa en varias librerías que permiten un código más ordenado y fácil de mantener.

II. Descripción de los módulos del STM32 usados

Para el desarrollo se utilizaron los módulos GPIO, USART1 y la memoria SRAM del STM32F103C8. GPIO permitió controlar el LED integrado en PC13, el cual se utilizó

para indicar cada modo. El módulo USART1 se configuró a 9600 bps para comunicarse con el HM-10. La SRAM alojó los arreglos buffer_m1 y buffer_m2, cada uno con 200 números generados mediante funciones pseudoaleatorias.

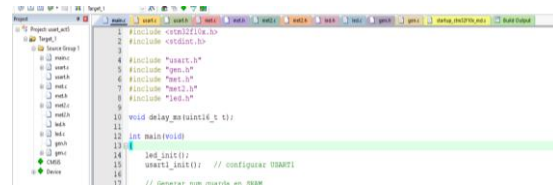


Figura 1. Proyecto General

III. Funcionamiento del código (explicado por el alumno)

Para organizar mejor el programa decidí dividir el código en varias librerías. En gen.c puse las funciones que generan los números aleatorios así no saturaba el main. En met.c coloqué el modo 1, que transmite los números de forma continua (sin espacios), y en met2.c el modo 2, que los envía en grupos de cinco. También agregué un archivo led.c para manejar el LED porque lo usé como indicador del funcionamiento. Finalmente, el archivo usart.c contiene la configuración básica del USART1, la cual tomé como referencia de la práctica anterior. Con todo eso, el

main solo llama a cada módulo y queda más limpio.

```
main.c | uart.c | uart.h | met.c | met.h
1 #include "stm32f10x.h"
2 #include "stdio.h"
3
4 #include "uart.h"
5 #include "gen.h"
6 #include "met.h"
7 #include "met2.h"
8 #include "led.h"
9
10 void delay_ms(unsigned int t);
11
12 int main(void)
13 {
14     led_init();
15     uart1_init(); // configurar USART1
16
17     // Generar num guarda en SRAM
18     generar_ml(); // buffer_ml(200)
19     generar_m2(); // buffer_m2(200)
20
21     // Llama modo 1
22     metodo1();
23
24     // Intermedio
25     delay_ms(1000);
26
27     // Llama modo 2
28     metodo2();
29
30     // Bucle infinito
31     while (1) {
32     }
33 }
34
35 // Retardo
36 void delay_ms(unsigned int t)
37 {
38     volatile unsigned long i;
39     while (t--) {
40         for (i = 0; i < 6000; i++) {
41         }
42     }
43 }
44
45 }
```

Figura 2. Organización del código main.

4. Solución propuesta y algoritmos usados

Se implementaron dos modos de operación. En el modo 1, el LED permanece encendido mientras se transmiten los 200 números sin separación.

```
Disconnect YAEL CLS Settings
Met 1
23652365236985076501850507698507214363
434947698107230505210781814363052347856
30185656349430189894363476927630149472
763076347818569498981010507692107272307
810523014369230143052723050507856501850
7814947
Fin 1

Met 2
61616
16723
49238
90941
85278
76107
09496
92505
27618
94121
29052
38545
89234
70721
67232
12505
23814
56927
87016
50549
41056
18181
47274
50169
49618
74383
-----
```

6. Conclusión

Gracias a esta actividad reforcé el uso de librerías, manejo de memoria SRAM y comunicación USART. Aprendí cómo funciona la transmisión hacia un módulo BLE y cómo organizar mi código para que sea más fácil de entender y modificar. La implementación de los modos con el LED también me ayudó a visualizar cómo se ejecuta cada parte del programa.

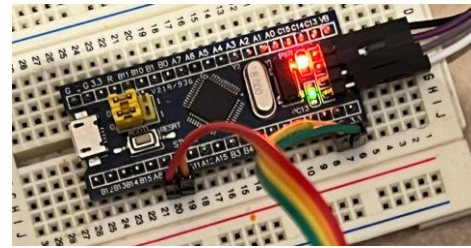
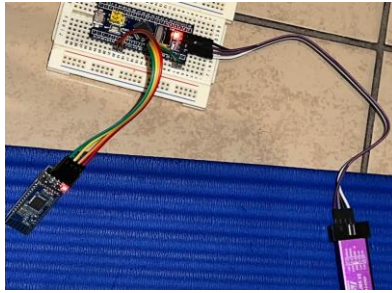
En el modo 2, el LED parpadea por cada grupo de cinco números enviados. Para generar los números utilicé un algoritmo lineal muy sencillo, donde multiplico la semilla por una constante y le sumo un valor fijo. Después hago módulo 10 para obtener un dígito entre 0 y 9. Esta técnica no es un RNG profesional, pero para la práctica funciona bien y es fácil de entender.

74383
23852
16145
01074
36965
47478
30727
43636
90749
89210
14169
61074
98927
49050
58547

Fin 2

5. Análisis de resultados

El sistema funcionó correctamente. Los datos se almacenaron en la SRAM como esperaba y la transmisión por Bluetooth se verificó usando la aplicación Tiny BLE Serial. El LED ayudó bastante a comprobar visualmente el funcionamiento de cada modo. La arquitectura modular también facilitó mucho identificar errores porque cada parte tenía su archivo.



7. Referencias

- STM32F103x8 Reference Manual
- Datasheet del módulo HM-10 BLE
- Apuntes del curso de Microprocesadores